

Building Scalable Kubernetes Governance

with Kyverno and CEL

Cansu Kavılı Örnek (Red Hat) & Koray Oksay (Kubermatic)

KCD Helsinki 2026



WHOAMI?

Koray Oksay

- Kubernetes Consultant and Trainer
- CNCF & Platform Engineering Ambassador
- KCD and DevOpsDays Istanbul Org.
- Contributor at #sig-k8s-infra and apisnoop/verify-conformance
- Maintaining CNCF Hosted GitHub Runners



WHOAMI?

Cansu Kavılı Örnek

- Principal Platform Architect @ Red Hat
- Based in Germany
- Platform Engineering Ambassador
- Team Topologies Advocate
- Huge believer in Open Source



"We just need a few policies..."



Require labels
on Deployments



Block "latest"
image tags



Enforce resource
requests & limits



Restrict privileged
containers

1 cluster · 1 team · 4 policies · Life is good.

...and then the platform grows.

5

clusters

12

teams

3

environments

60+

policy files

-
- ⚡ Copy-paste YAML everywhere
 - ⚡ "Almost identical" policies that drift apart
 - ⚡ One-off exceptions become permanent snowflakes
 - ⚡ PRs that touch 15 files to change one label rule

From static rules → dynamic guardrails



Write once, apply contextually

One policy that behaves differently per namespace, team, or environment.



Externalize configuration

Policy logic in Kyverno, policy data in your cluster. Update config, not code.



Express complex invariants simply

Cross-field validation, string logic, list operations — without YAML gymnastics.

CEL in 60 Seconds

Common Expression Language — lightweight, safe, fast, extendible.

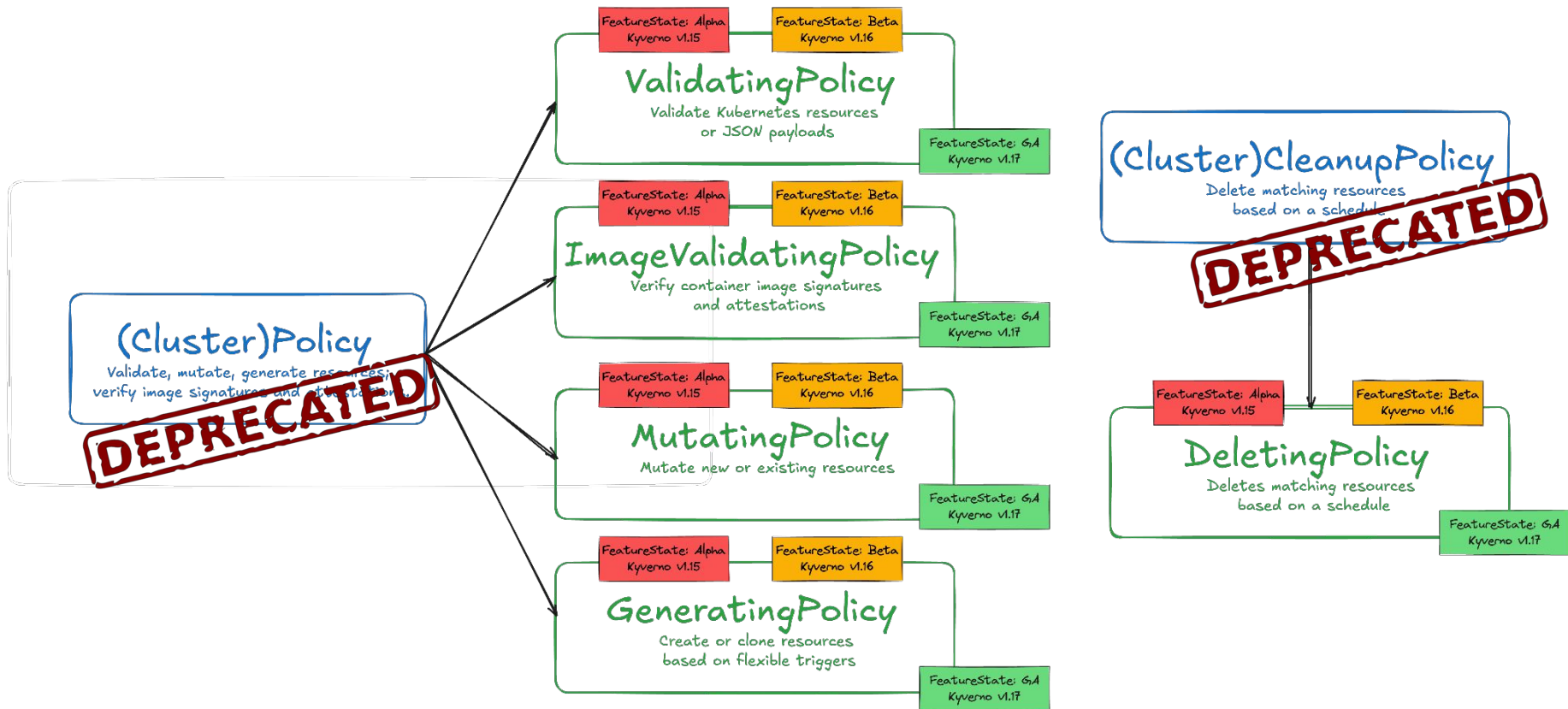
```
// Boolean logic
object.spec.replicas > 1

// String operations
object.metadata.name.startsWith("prod-")

// List comprehensions
object.spec.template.spec.containers.all(c,
    has(c.resources)
)

// Optional chaining
object.metadata.labels['?tier'].orValue('dev')
```

Kyverno Policies Recap



Deprecation Schedule for Legacy Types

Release	Date (Estimated)	Status
v1.17	Jan 2026	Marked for deprecation
v1.18	Apr 2026	Critical fixes only
v1.19	Jul 2026	Critical fixes only
v1.20	Oct 2026	Planned for removal

ValidatingPolicy with CEL

```
apiVersion: policies.kyverno.io/v1
kind: ValidatingPolicy
metadata:
  name: require-owner-in-prod
spec:
  matchConstraints:
    resourceRules:
      - apiGroups: ["apps"]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources: ["deployments"]
  variables:                                     # ← define context
    - name: isProduction
      expression: >-
        has(namespaceObject.metadata.labels) &&
        namespaceObject.metadata.labels[?'env']
          .orValue('dev') == 'production'
  validations:                                  # ← CEL expressions
    - expression: >-
        !variables.isProduction ||
        has(object.metadata.labels.owner)
      message: "Prod deployments need an owner label."
```

Pattern 1

Namespace-Driven Context

```
variables:
  - name: env
    expression: >-
      namespaceObject.metadata
        .labels[?'env'].orValue('dev')
validations:
  - expression: |
      variables.env != 'production' ||
      object.spec.template.spec
        .containers.all(c,
          c.image.contains(':') &&
          !c.image.endsWith(':latest'))
    messageExpression: |
      "Namespace env is '"
      + variables.env + "'. ..."
```

What happens



nginx:latest in prod
→ DENIED



nginx:latest in dev
→ ALLOWED



nginx:1.25.3 in prod
→ ALLOWED

Pattern 2

Externalized Configuration

```
# ConfigMap – the data
apiVersion: v1
kind: ConfigMap
metadata:
  name: registry-policy-config
  namespace: kyverno
data:
  allowed-registries: >-
    gcr.io/myorg,docker.io/mycompany
```

```
# Policy variable – reads the ConfigMap
variables:
- name: allowedRegistries
  expression: |
    resource.Get("v1", "configmaps", "kyverno",
      "registry-policy-config")
    .data["allowed-registries"]
    .split(",")
```

The decoupling



Policy logic
lives in Kyverno



Policy config
lives in a ConfigMap



Update the ConfigMap
→ behavior changes



No PR/redeployment for the policy.
Just a ConfigMap edit

Pattern 3

Expressive Invariants

```
validations:
- expression: |
    object.spec.replicas <= 1 ||
    has(object.spec.template.spec
      .affinity) &&
    has(object.spec.template.spec
      .affinity.podAntiAffinity)
messageExpression: |
  "Deployment '"
  + object.metadata.name
  + "' has "
  + string(object.spec.replicas)
  + " replicas but no "
  + "podAntiAffinity."
```

What happens



3 replicas, no affinity
→ DENIED



3 replicas + anti-affinity
→ ALLOWED



1 replica, no affinity
→ ALLOWED

*Pure CEL logic — no lookups,
no ConfigMaps, just expressions*



Live Demo

Three patterns · One Kind cluster · Real policies

What We Just Saw

1

Namespace-Aware Image Tags

One policy adapts to env labels — replaces per-namespace copies.

variables +
namespaceObject

2

ConfigMap-Driven Registry Allowlist

Policy logic stays fixed. Platform teams update a ConfigMap.

get() cluster state
lookup

3

Replicas + Anti-Affinity

Complex invariant in a single CEL expression. No YAML gymnastics.

cross-field
validation

Pattern 1

Namespace-Driven Context

Write once, apply contextually

- ✓ Label namespaces with env, tier, team
- ✓ Use namespaceObject in CEL to branch logic
- ✓ One ClusterPolicy replaces N per-env copies

```
variables:  
  - name: env  
    expression: "namespaceObject.metadata.labels[?'env'].orValue('dev')"  
validations:  
  - expression: "variables.env != 'production' || <your rule>"
```

Pattern 2

Externalized Configuration

Decouple policy logic from policy data

- ✓ Store exception lists, registries, thresholds in ConfigMaps (or other Kubernetes objects)
- ✓ Reference via resource.Get() in CEL variables
- ✓ Platform teams update config without touching policies

```
variables:  
  - name: allowedRegistries  
    expression: |  
      resource.Get("v1", "configmaps", "kyverno", "registry-config")  
      .data["allowed-registries"].split(",")
```

Pattern 3

Expressive Invariants

Say what you mean, not how to match it

```
replicas > 1 → podAntiAffinity required
```

HA enforcement

```
limits >= requests for CPU & memory
```

Resource sanity

```
image.matches('^v[0-9]+\.[0-9]+\.[0-9]+$')
```

```
expression: |
  object.spec.replicas <= 1 ||
  has(object.spec.template.spec.affinity) &&
  has(object.spec.template.spec.affinity.podAntiAffinity)
```

Before & After

	YAML-Only	Kyverno + CEL
Policy files	60+ across clusters	8–10 parameterized
Adding an env	Copy & tweak all files	Add namespace labels
New team onboard	3–5 PRs	1 namespace manifest
Exception handling	Fork the policy	Update a ConfigMap
Readability	Nested YAML overlays	if prod then require X

Getting Started

- 1 Label your namespaces with governance metadata
- 2 Pick your top 3 noisiest policies — rewrite with CEL
- 3 Run in Audit mode first — policy reports show what would fail
- 4 Iterate — add variables, consolidate, remove old YAML



Kyverno + CEL: policy-as-code that actually scales

Thank you!



cansu@redhat.com



[linkedin.com/in/ckavili](https://www.linkedin.com/in/ckavili)



koray@kubermatic.com



[linkedin.com/in/korayoksay](https://www.linkedin.com/in/korayoksay)